ARKA-Lite

Proyecto final · Upskilling Backend Java

Una UI y dos microservicios Spring Boot: lo que aprendimos y cómo funciona.

RECORRIDO EJECUTIVO · CONTENIDO Y GUION

REPARTO · EQUIPO ARKA-LITOS

Participante 1 · Desarrollo Apertura, Sistema, Cierre
Participante 2 · Desarrollo Servicio
Participante 3 · Desarrollo Diseño
Participante 4 · Base de datos Datos
Participante 5 · DevOps Empaquetar
Participante 6 · DevOps Operar
Participante 7 · Desarrollo UI

Generado desde docs/presentation-data/contenido. Cada lámina tiene: lo que se ve, el diagrama paso a paso cuando lo hay, y el guion: qué decir, qué hacer, si preguntan y para pasar.

Apertura

Presenta: Participante 1 · Desarrollo

1. Del endpoint al clúster

EN PANTALLA

- Cómo construimos ARKA-Lite: una interfaz web y dos microservicios en Java.

QUÉ DECIR

“Buenos días a todos. Somos el equipo arka-litos, del curso Upskilling Backend Java, y hoy les vamos a contar cómo construimos ARKA-Lite, nuestro proyecto final.”

“El título lo resume: empezamos escribiendo un endpoint, una sola operación en Java, y terminamos con un sistema completo corriendo en un clúster de Kubernetes. En el camino fuimos aplicando, uno por uno, los temas que vimos en el curso.”

QUÉ HACER (NO SE DICE)

- Antes de empezar, abrir la UI en http://localhost:8088 y comprobar que los dos servicios aparecen en verde.

PARA PASAR

“Primero, quiénes somos.”

2. El equipo arka-litos

EN PANTALLA · EL EQUIPO POR ÁREA

- Desarrollo (4): Participante 1, Participante 2, Participante 3, Participante 7
- DevOps (2): Participante 5, Participante 6
- Base de datos (1): Participante 4

IDEAS CLAVE

- Cuatro personas de desarrollo
- Dos personas de DevOps
- Una persona de base de datos
- Cada quien presenta la parte de su área

QUÉ DECIR

“Somos siete personas. Cuatro somos de desarrollo: construimos los servicios, su diseño y la interfaz. Dos son de DevOps: se encargaron de empaquetar, desplegar y operar el sistema. Y una es de base de datos: cuidó cómo se guardan los datos y que ningún aviso se pierda.”

“Por eso hoy van a escuchar varias voces: cada uno presenta la parte de su área. Debajo de cada persona ven qué partes le tocan.”

“La idea no es que memoricen tecnologías. Queremos que, al salir, puedan explicar con sus propias palabras cómo funciona el sistema y por qué tomamos cada decisión.”

PARA PASAR

“Empecemos por lo básico: qué hace ARKA-Lite.”

---

3. Qué es ARKA-Lite

EN PANTALLA

- Solicitudes: crea solicitudes y cambia su estado
- Notificaciones: crea un aviso cuando una solicitud se envía
- La interfaz web: permite usar y entender los dos

Comparación: Como una ventanilla y un buzón: la ventanilla recibe el trámite y el buzón te avisa cuando avanzó.

QUÉ DECIR

“ARKA-Lite hace algo sencillo: registra solicitudes, por ejemplo una incidencia o un pedido de cambio, y avisa cuando una solicitud se envía.”

“Por dentro son dos servicios. El de solicitudes es como la ventanilla: recibe el trámite y lleva su estado. El de notificaciones es como el buzón: cuando el trámite avanza, deja el aviso. Y encima de los dos hay una interfaz web, que es lo que usa la persona.”

“Elegimos un problema pequeño a propósito. Así el foco no está en el negocio, sino en cómo se construye, se despliega y se opera un sistema con más de un servicio.”

PARA PASAR

“Antes de entrar en detalle, veamos el proyecto en números.”

---

4. ARKA-Lite en números

EN PANTALLA · NÚMEROS GRANDES

- 2 microservicios: Spring Boot con Java 26, cada uno con su base PostgreSQL.
- 11 endpoints REST: 8 en solicitudes y 3 en notificaciones, documentados con Swagger.
- 8 contenedores: Servicios, bases, interfaz y logs, con un solo comando.
- 67 pruebas automáticas: 28 en solicitudes, 10 en notificaciones y 29 en la interfaz.
- 5 s para entregar un aviso: Cada cinco segundos se entregan los avisos pendientes.
- 0 avisos duplicados: Aunque un evento llegue dos veces, se crea un solo aviso.

IDEAS CLAVE

- Dos microservicios y una interfaz
- Todo se levanta con un solo comando
- Cada cambio pasa por pruebas automáticas

QUÉ DECIR

“Así se ve ARKA-Lite en números. Son dos microservicios hechos con Spring Boot, cada uno con su propia base de datos, y entre los dos exponen once endpoints REST.”

“Todo el sistema son ocho contenedores: los servicios, sus bases, la interfaz y las herramientas de logs. Y se levantan con un solo comando.”

“Lo cuidan sesenta y siete pruebas automáticas. Los avisos pendientes se entregan cada cinco segundos y, aunque un evento llegue dos veces, se crea un solo aviso. Esos dos números los vamos a explicar con calma más adelante.”

PARA PASAR

“¿Y cómo se conecta esto con lo que vimos en el curso?”

5. Cómo vamos a recorrerlo

EN PANTALLA

- Siete partes, en el orden en que se construye
- Cada tema del curso, donde se usa
- Diagramas paso a paso
- Demostración en vivo al final

Parte	Temas del curso que aparecen
1 El sistema por fuera	Visión general del sistema y su arquitectura
2 Construir un servicio	Spring Boot, endpoints REST, manejo global de errores, Swagger, pruebas, Git
3 Diseñar para cambiar	Puertos y adaptadores, de monolito a microservicios
4 Guardar sin perder datos	Adaptador con base de datos real, el problema de la consistencia
5 Empaquetar y entregar	Docker y Dockerfile, Docker Compose, integración y entrega continua
6 Desplegar y operar	Kubernetes, del contenedor al clúster, puesta a punto, logs y monitoreo
7 Ensamblaje	Ensamblaje del sistema y puesta a punto

QUÉ DECIR

“No vamos a seguir el temario clase por clase. Vamos a contar la historia de cómo se construye el sistema, en el orden en que tiene sentido.”

“Primero lo vemos por fuera, como un usuario. Después entramos a un servicio y vemos cómo se arma. Luego, cómo lo diseñamos para que pueda cambiar sin romperse y cómo cuidamos los datos. Nuestros compañeros de DevOps nos muestran cómo lo empaquetamos, lo desplegamos y sabemos si está funcionando. Al final lo vemos todo junto, en vivo.”

“En cada parte van a ver el mismo orden: qué problema resuelve, cómo lo aplicamos, un diagrama paso a paso y la prueba de que funciona.”

QUÉ HACER (NO SE DICE)

- Explicar que el número sobre cada flecha de los diagramas coincide con el paso que se lee.

PARA PASAR

“Empecemos mirando el sistema completo desde afuera.”

1 · El sistema por fuera

Presenta: Participante 1 · Desarrollo

6. Portada · El sistema por fuera

EN PANTALLA

- ¿Qué problema resuelve? Antes de ver código, hay que saber qué piezas existen y cómo se hablan.
- ¿Cómo funciona? Una UI llama a dos microservicios por HTTP; cada uno guarda sus datos en su propia base.
- ¿Cómo se ve? Persona -> UI -> solicitudes y notificaciones -> una base PostgreSQL para cada uno.
- ¿Dónde está en ARKA-Lite? compose.yaml levanta la UI, los dos servicios, sus bases y la observabilidad con un solo comando.
- ¿Cómo lo comprobamos? La UI en localhost:8088 con los dos servicios en verde.

QUÉ DECIR

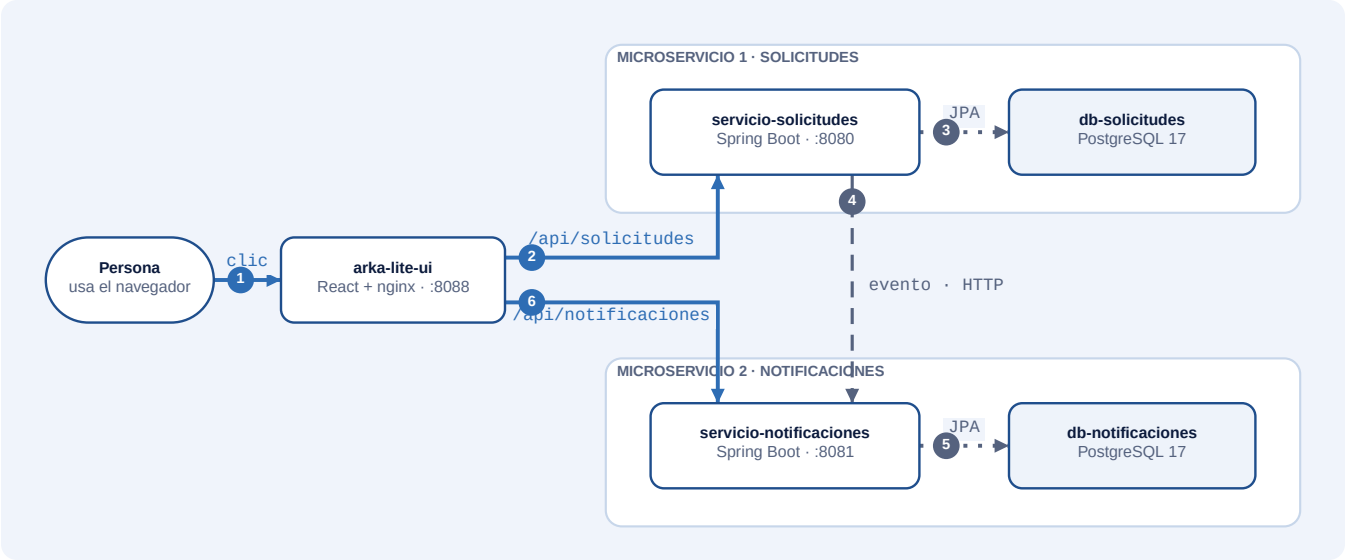
“Ahora vamos con el sistema por fuera. Primero, qué problema resuelve.”

7. El mapa de ARKA-Lite

EN PANTALLA

- La UI es la puerta de entrada
- Cada servicio tiene su propia base
- Solicitudes avisa a notificaciones con un evento
- Todo se comunica por HTTP

DIAGRAMA PASO A PASO · EL MAPA DE ARKA-LITE



Petición · · · · Paso interno explicado — — Trabajo posterior

1. La persona usa la UI en el navegador.
2. La UI llama a solicitudes por /api: crear, listar, enviar, aprobar o rechazar.
3. Solicitudes guarda los datos en su propia base.
4. Al enviar, solicitudes avisa después a notificaciones con un evento por HTTP.
5. Notificaciones guarda en su base qué eventos ya consumió.
6. La UI también consulta a notificaciones: avisos y eventos consumidos.

Cada servicio es dueño de su base. Ninguno lee la base del otro.

QUÉ DECIR

"Este es el mapa. A la izquierda está la persona, usando la interfaz desde su navegador. La interfaz no guarda nada: le pide las cosas a los servicios."

"Arriba está el servicio de solicitudes, con su base de datos. Abajo, el de notificaciones, con la suya. Fíjense en algo importante: ninguno lee la base del otro. Si notificaciones necesita saber algo, solicitudes se lo cuenta con un mensaje que llamamos evento."

"Y todo viaja por HTTP, el mismo protocolo que usa cualquier página web. No hay magia: son peticiones y respuestas."

SI PREGUNTAN

¿Por qué no comparten una sola base de datos?

Para que cada servicio sea dueño de sus datos y pueda cambiar sin romper al otro.

PARA PASAR

"Veamos qué le pasa a una solicitud durante su vida."

8. La vida de una solicitud

EN PANTALLA

- Nace en BORRADOR
- Al enviarla pasa a ENVIADA y se genera un aviso
- Desde ENVIADA se aprueba o se rechaza
- Un cambio no permitido responde 409

Acción	Desde	Hacia	¿Genera aviso?
Crear	—	BORRADOR	No
Enviar	BORRADOR	ENVIADA	Sí, un evento para notificaciones
Aprobar	ENVIADA	APROBADA	No
Rechazar	ENVIADA	RECHAZADA	No

QUÉ DECIR

"Toda solicitud nace en borrador. Mientras está en borrador, nadie se entera: es como un formulario que todavía no hemos entregado."

"Cuando la enviamos, pasa a enviada, y ese es el único momento en que se genera un aviso. Desde enviada, alguien la puede aprobar o rechazar, y ahí termina su vida."

"¿Qué pasa si intentamos algo que no corresponde, como enviar dos veces la misma solicitud? El servicio no lo permite y responde con un código 409, que en HTTP significa conflicto, junto con un mensaje que explica el porqué."

QUÉ HACER (NO SE DICE)

- Mostrar: En la UI, pestaña Solicitudes: crear una solicitud nueva y enviarla. Debe verse: La fila pasa de Borrador a Enviada y, unos segundos después, muestra 1 aviso.

PARA PASAR

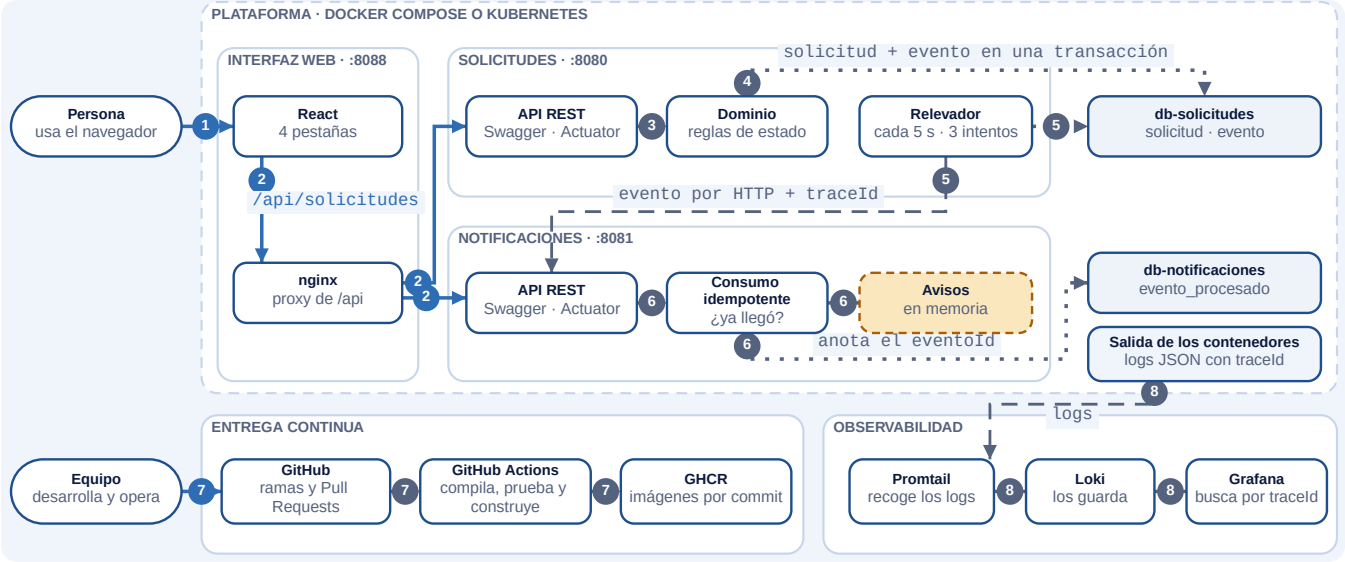
"Esa es la vista sencilla. Ahora veamos todo lo que hay alrededor."

9. La arquitectura completa

EN PANTALLA

- Las piezas corren en Docker Compose o en Kubernetes
- GitHub Actions prueba cada cambio y publica las imágenes
- Los logs de todos llegan a Grafana
- Cada pieza tiene una sola responsabilidad

DIAGRAMA PASO A PASO · LA ARQUITECTURA COMPLETA



Petición · · · · Paso interno explicado — — Trabajo posterior

1. La persona usa la interfaz en su navegador: React muestra las cuatro pestañas.
2. nginx recibe /api y reenvía cada ruta al servicio que corresponde.
3. La API de solicitudes recibe la petición y el dominio aplica las reglas.
4. En una sola transacción se guardan la solicitud y su evento pendiente.
5. Cada 5 segundos, el relevador lee los pendientes y los entrega por HTTP, con el traceId.
6. Notificaciones revisa si el evento ya llegó; si es nuevo, crea el aviso y anota su número.
7. Cada cambio en GitHub pasa por GitHub Actions, que prueba y publica las imágenes en GHCR.
8. Los logs de todos los contenedores llegan a Grafana a través de Promtail y Loki.

Cada servicio es dueño de su base. Ninguno lee la base del otro.

QUÉ DECIR

"Este es el mismo sistema, pero con todo lo que lo rodea. El recuadro grande es el lugar donde corre: en nuestra computadora con Docker Compose, o en un clúster de Kubernetes."

"Abajo a la izquierda está la entrega continua. Cada vez que alguien sube un cambio a GitHub, un proceso automático lo compila, corre las pruebas y, si todo sale bien, publica las imágenes listas para desplegar."

"Y abajo a la derecha, la observabilidad: los logs de todos los contenedores se recogen con Promtail, se guardan en Loki y se consultan en Grafana. Así, si algo falla, no tenemos que entrar a cada servicio a buscar qué pasó."

SI PREGUNTAN

Si mañana agregamos un tercer servicio, ¿qué partes del dibujo cambian?

Se agrega su caja y su base; la entrega y los logs lo cubren igual, sin cambios.

PARA PASAR

"Ya vimos el mapa. Ahora entremos a uno de esos servicios."

2 · Construir un servicio

Presenta: Participante 2 · Desarrollo

10. Portada · Construir un servicio

EN PANTALLA

- ¿Qué problema resuelve? Necesitamos exponer operaciones del negocio por HTTP de forma clara, documentada y probada.
- ¿Cómo funciona? Spring Boot arma la aplicación; los controladores reciben peticiones; el dominio decide; las pruebas lo comprueban.
- ¿Cómo se ve? Petición -> controlador -> servicio -> repositorio -> respuesta con su código HTTP.
- ¿Dónde está en ARKA-Lite? SolicitudController, ServicioDeSolicitudes, ManejadorGlobalDeErrores, Swagger y las pruebas JUnit.
- ¿Cómo lo comprobamos? ./mvnw verify en cada servicio y Swagger en /swagger-ui/index.html.

QUÉ DECIR

“Ahora vamos con construir un servicio. Primero, qué problema resuelve.”

11. Primer contacto con Spring Boot

EN PANTALLA

- Maven descarga lo necesario, compila y prueba
- Spring Boot levanta el servidor y conecta las piezas
- Cada pieza recibe lo que necesita por el constructor
- Las reglas del negocio no dependen de Spring

Comparación: Como un contratista de obra: nosotros decimos qué piezas hay y él las arma y las conecta.

QUÉ DECIR

“Nuestro primer contacto con Spring Boot fue entender qué hace por nosotros. Maven se encarga de descargar las librerías, compilar y correr las pruebas. Spring Boot levanta el servidor web y conecta las piezas de la aplicación.”

“Piénsenlo como un contratista de obra: nosotros le decimos qué piezas hay, y él las arma y las conecta. Cada pieza recibe lo que necesita por su constructor; nadie sale a buscar sus dependencias por su cuenta.”

“Una decisión que tomamos desde el principio: las reglas del negocio no conocen a Spring. Son clases normales de Java. Eso nos va a servir mucho cuando hablemos de pruebas y de diseño.”

QUÉ HACER (NO SE DICE)

- Mostrar: Ejecutar ./mvnw clean verify dentro de servicio-solicitudes. Debe verse: BUILD SUCCESS y el resumen de pruebas.

PARA PASAR

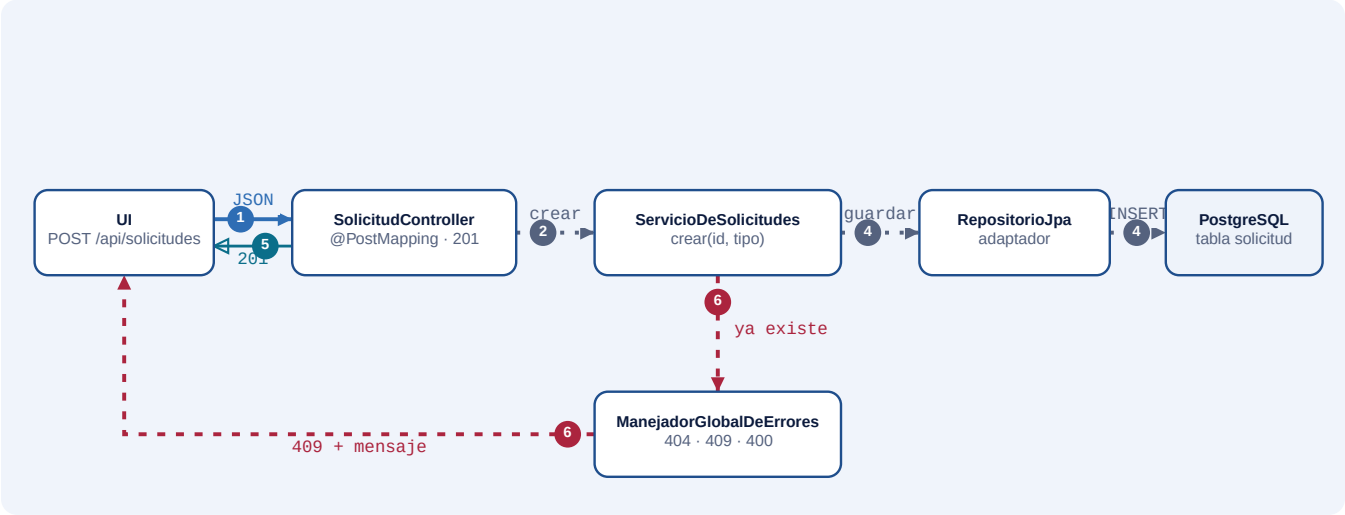
“Con la aplicación arriba, construyamos nuestro primer endpoint.”

12. Construcción de endpoints REST

EN PANTALLA

- Método + ruta = una operación
- El cuerpo viaja en JSON
- Cada respuesta trae un código: 201, 404, 409
- El controlador traduce; quien decide es el servicio

DIAGRAMA PASO A PASO · CREAR UNA SOLICITUD POR REST



1. La UI envía POST con un cuerpo JSON: id y tipo.
2. El controlador convierte el JSON en NuevaSolicitud y llama al servicio.
3. El servicio valida los datos y revisa que el id no exista.
4. El adaptador JPA guarda la fila en PostgreSQL.
5. La API responde 201 Created con la solicitud en BORRADOR.
6. Si el id ya existe, el manejador global de errores responde 409 con un mensaje claro.

Los mismos contratos se ven en Swagger: http://localhost:8080/swagger-ui/index.html

QUÉ DECIR

“Un endpoint REST es una combinación de método y ruta. Por ejemplo, POST a /solicitudes significa: crea una solicitud nueva. GET a /solicitudes significa: dame la lista.”

“Sigamos el diagrama. La interfaz manda los datos en formato JSON. El controlador los recibe y se los pasa al servicio. El servicio revisa que todo esté bien y el adaptador lo guarda en la base de datos. Si todo sale bien, la respuesta es 201, que quiere decir creado.”

“Y si el identificador ya existía, el servicio no lo guarda y responde 409, conflicto, con un mensaje claro. Cada código cuenta qué pasó, sin tener que adivinar.”

PARA PASAR

“¿Y dónde vive la regla que decide si algo se permite?”

13. La regla vive en el dominio

EN PANTALLA

- La solicitud es un record: datos que no cambian
- Solo se envía si está en BORRADOR
- Si no, lanza una excepción del dominio
- No sabe nada de HTTP ni de bases de datos

QUÉ DECIR

“La solicitud es un record de Java, que es una forma corta de declarar datos que no cambian. Cuando la enviamos, no modificamos la solicitud que teníamos: creamos una nueva en estado enviada.”

“Y la regla está ahí mismo, a la vista: solo se puede enviar si está en borrador. Si no, lanza una excepción que se llama TransicionNoPermitida.”

“Fíjense en lo que esta clase no sabe: no sabe de HTTP, no sabe de PostgreSQL. Solo sabe del negocio. Si mañana cambia la regla, la cambiamos en un solo lugar.”

PARA PASAR

“Esa excepción tiene que llegarle a la persona como un mensaje que entienda.”

14. Manejo global de errores

EN PANTALLA · NÚMEROS GRANDES

- 201 Creada: Todo salió bien: la solicitud quedó en BORRADOR.
- 400 Faltan datos: El identificador o el tipo vienen vacíos.
- 404 No existe: No hay ninguna solicitud con ese identificador.
- 409 Conflicto: El identificador ya existe o el cambio no se permite.

IDEAS CLAVE

- 404: la solicitud no existe
- 409: el identificador ya existe o el cambio no se permite
- 400: faltan datos
- Siempre la misma forma: estado y mensaje

QUÉ DECIR

“En el taller del endpoint seguro aprendimos a no repetir el manejo de errores en cada controlador. En su lugar, hay un solo componente, el manejador global, que atrapa las excepciones y las convierte en respuestas HTTP.”

“Si la solicitud no existe, responde 404. Si el identificador ya existe o el cambio no se permite, 409. Si faltan datos, 400. Y siempre con la misma forma: un estado y un mensaje.”

“Eso es lo que permite que la interfaz le muestre a la persona un mensaje claro, en lugar de un error genérico que nadie entiende.”

QUÉ HACER (NO SE DICE)

- Mostrar: En la UI, crear dos veces el mismo identificador. Debe verse: Aparece el diálogo “Ese identificador ya existe” con el código 409.

PARA PASAR

“Ahora, ¿cómo sabe otra persona qué endpoints existen?”

---

15. Documentar con Swagger

EN PANTALLA

- Cada servicio publica su contrato con OpenAPI
- Swagger lo muestra como una página
- Se puede probar cada endpoint desde ahí
- Si agregamos un endpoint, aparece solo

Comparación: Como el menú de un restaurante: dice qué hay, qué lleva cada plato y qué te van a traer.

Servicio	Swagger
solicitudes	http://localhost:8080/swagger-ui/index.html
notificaciones	http://localhost:8081/swagger-ui/index.html

QUÉ DECIR

“Un contrato de API es como el menú de un restaurante: dice qué se puede pedir, qué lleva cada pedido y qué te van a devolver.”

“Cada servicio publica su contrato con un estándar llamado OpenAPI, y Swagger lo muestra como una página donde podemos ver cada endpoint y hasta probarlo.”

“Lo mejor es que se genera desde el código. Cuando agregamos un endpoint, la documentación lo muestra sin trabajo extra, y nunca queda desactualizada. Desde la interfaz hay un menú, Contratos, que abre estas páginas directamente.”

QUÉ HACER (NO SE DICE)

- Mostrar: En la UI, menú Contratos ? Swagger · solicitudes, y ejecutar GET /solicitudes. Debe verse: La lista en JSON con código 200.

PARA PASAR

“Documentar no basta: también hay que comprobar que funciona.”

16. Pruebas unitarias

EN PANTALLA

- Las reglas se prueban sin levantar Spring
- Los controladores se prueban con MockMvc
- Un repositorio en memoria reemplaza a PostgreSQL
- El pipeline corre todas las pruebas en cada cambio

Prueba	Qué comprueba
SolicitudTest	Los cambios de estado permitidos y prohibidos
ServicioSolicitudesUnitTest	Crear, enviar y registrar el evento
SolicitudControllerWebTest	El JSON y los códigos 201, 404, 409 y 400
Pruebas de notificaciones	Un evento repetido genera un solo aviso

QUÉ DECIR

“Las pruebas son como un vigilante que revisa cada regla todas las noches. Si alguien cambia algo y rompe una regla sin darse cuenta, la prueba falla y nos avisa.”

“Como el negocio no depende de Spring, podemos probar las reglas sin levantar nada: son pruebas muy rápidas. En lugar de PostgreSQL usamos un repositorio en memoria que cumple el mismo contrato.”

“Los controladores los probamos con MockMvc, que simula peticiones HTTP sin un servidor real. Solo el servicio de solicitudes tiene 28 pruebas, y el pipeline las corre todas en cada cambio.”

QUÉ HACER (NO SE DICE)

- Mostrar: ./mvnw test en servicio-solicitudes. Debe verse: Tests run: 28, Failures: 0.

PARA PASAR

“Las pruebas cuidan el código. Git cuida su historia.”

17. Git y commits con sentido

EN PANTALLA · UNA RAMA DE GIT

- Rama feat/crear-solicitud-por-post sale de main
- Commit: feat: crear solicitud por post
- Commit: fix: responder 409 si el id ya existe
- Commit: docs: documentar POST /solicitudes
- Vuelve a main: Pull Request: una aprobación y el pipeline en verde

IDEAS CLAVE

- Una rama corta por tarea: feat/, fix/, docs/
- Commits en español: “feat: crear solicitud por post”

- Todo entra por Pull Request con una aprobación
- Nadie sube cambios directo a main

QUÉ DECIR

“Trabajamos en equipo sobre el mismo código, así que necesitábamos orden. Cada tarea se hace en una rama corta, con un nombre que dice qué es: feat para algo nuevo, fix para una corrección, docs para documentación.”

“Cada commit sigue una convención: el tipo, dos puntos y una frase corta que dice qué hace el cambio. Por ejemplo: feat, crear solicitud por post. Así, leyendo el historial, cualquiera entiende qué pasó.”

“Y nadie sube cambios directo a la rama principal. Todo entra por un Pull Request, otra persona lo revisa y el pipeline lo valida antes de integrarlo.”

QUÉ HACER (NO SE DICE)

- Mostrar el historial del repositorio en GitHub; no crear commits durante la exposición.

PARA PASAR

“Ya sabemos construir un servicio. Ahora veamos cómo lo diseñamos para que pueda cambiar.”

3 · Diseñar para cambiar

Presenta: Participante 3 · Desarrollo

18. Portada · Diseñar para cambiar

EN PANTALLA

- ¿Qué problema resuelve? Si las reglas dependen de HTTP o de la base, cambiar una tecnología obliga a tocar el negocio.
- ¿Cómo funciona? El negocio va en el centro y la tecnología en los bordes. Lo que cambia por motivos distintos va en servicios distintos.
- ¿Cómo se ve? Entrada -> dominio -> salida; un monolito que se separa en dos servicios.
- ¿Dónde está en ARKA-Lite? Paquetes dominio/ y adaptadores/ en cada servicio; adaptadores activos marcados con @Primary.
- ¿Cómo lo comprobamos? Las pruebas usan un repositorio en memoria con el mismo puerto, sin PostgreSQL.

QUÉ DECIR

“Ahora vamos con diseñar para cambiar. Primero, qué problema resuelve.”

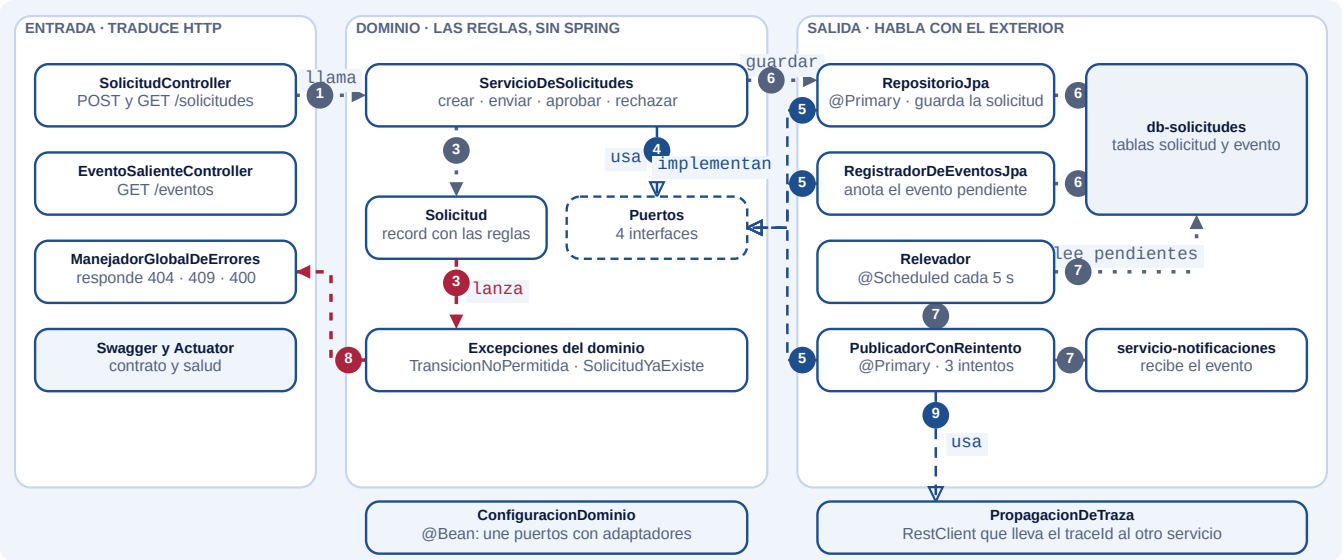
19. Por dentro de servicio-solicitudes

EN PANTALLA

- Entrada: traduce HTTP
- Dominio: las reglas, sin Spring

- Puertos: lo que el negocio necesita
- Salida: PostgreSQL y HTTP

DIAGRAMA PASO A PASO · POR DENTRO DE SERVICIO-SOLICITUDES



1. Las peticiones entran por el controlador, que solo traduce HTTP.
2. El servicio de dominio coordina cada caso: crear, enviar, aprobar o rechazar.
3. Las reglas viven en el record Solicitud; si una no se cumple, lanza una excepción.
4. El dominio pide lo que necesita a través de puertos: interfaces sin tecnología.
5. Los adaptadores de salida implementan esos puertos; Spring usa los marcados con @Primary.
6. Al enviar, la solicitud y su evento pendiente se guardan en PostgreSQL.
7. Cada 5 segundos, el relevador lee los pendientes y el publicador los entrega con reintentos.
8. El manejador global convierte las excepciones en 404, 409 o 400; Swagger y Actuator publican el contrato y la salud.
9. ConfiguracionDominio arma las piezas con @Bean, y el RestClient lleva el traceId al otro servicio.

Como el dominio no conoce la tecnología, las pruebas lo usan con un repositorio en memoria.

QUÉ DECIR

"En el taller de estructura base organizamos cada servicio en tres zonas. A la izquierda, la entrada: los controladores que reciben las peticiones HTTP. En el centro, el dominio: las reglas del negocio. A la derecha, la salida: lo que habla con el exterior, como la base de datos o el otro servicio."

"El dominio no llama directamente a la base. Pide lo que necesita a través de puertos, que son interfaces: un contrato que dice, por ejemplo, necesito guardar y buscar solicitudes, sin decir cómo."

"Los adaptadores de salida cumplen ese contrato. Uno guarda en PostgreSQL; otro anota el evento pendiente; otro lo entrega a notificaciones. A esto se le llama arquitectura de puertos y adaptadores, o arquitectura hexagonal."

PARA PASAR

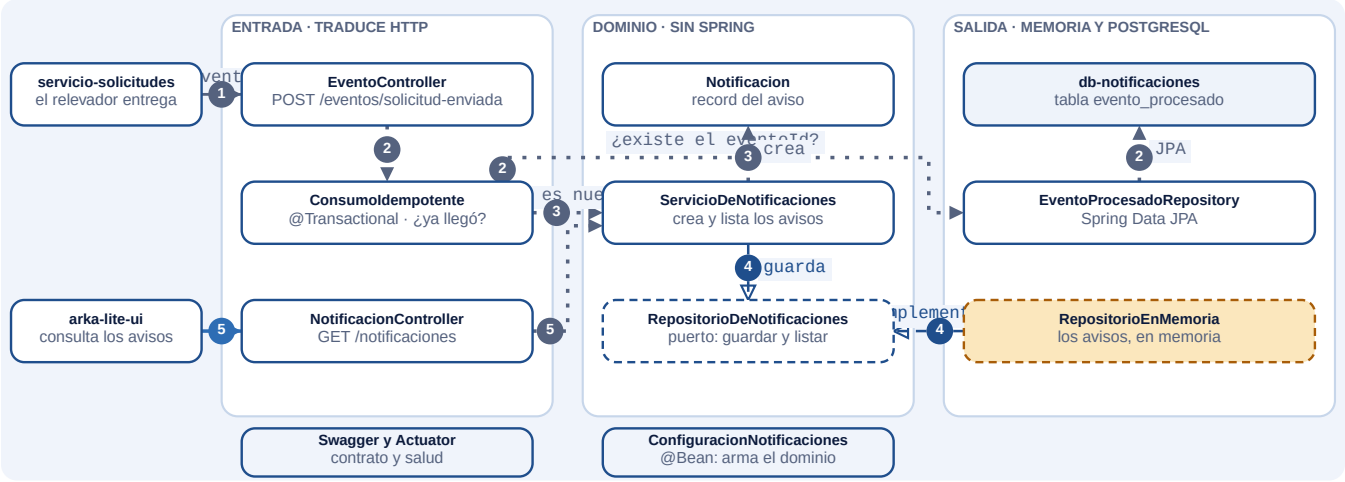
"El otro servicio sigue la misma idea."

20. Por dentro de servicio-notificaciones

EN PANTALLA

- Entrada: recibe el evento y las consultas
- Consumo idempotente: ¿este evento ya llegó?
- Dominio: crea el aviso
- Salida: avisos en memoria y eventos en PostgreSQL

DIAGRAMA PASO A PASO · POR DENTRO DE SERVICIO-NOTIFICACIONES



1. Solicitudes entrega el evento por HTTP a EventoController.
2. ConsumoIdempotente pregunta si ese eventoId ya está en evento_procesado.
3. Si es nuevo, el dominio crea el aviso; si ya estaba, lo ignora.
4. El aviso se guarda en memoria por el puerto, y el eventoId queda anotado en PostgreSQL.
5. La interfaz consulta los avisos con GET /notificaciones.
6. ConfiguracionNotificaciones arma el dominio; Swagger y Actuator publican el contrato y la salud.

El texto de los avisos vive en memoria; la lista de eventos recibidos, en PostgreSQL.

QUÉ DECIR

"Ahora el otro servicio. Notificaciones es más pequeño, pero está organizado igual: entrada, dominio y salida."

"Por la entrada llegan dos cosas: el evento que manda solicitudes y las consultas de la interfaz. Antes de crear un aviso, una pieza que llamamos consumo idempotente revisa en la base si ese número de evento ya llegó. Si es nuevo, el dominio crea el aviso y se anota el número."

"Y un detalle honesto: los avisos viven en memoria, pero la lista de eventos ya recibidos está en PostgreSQL. Por eso, aunque el servicio se reinicie, nunca crea un aviso repetido."

SI PREGUNTAN

¿Qué pasa con los avisos si notificaciones se reinicia?

Se pierde su texto, pero no se duplican: la lista de eventos recibidos está en la base.

PARA PASAR

“¿Qué ganamos con esa separación?”

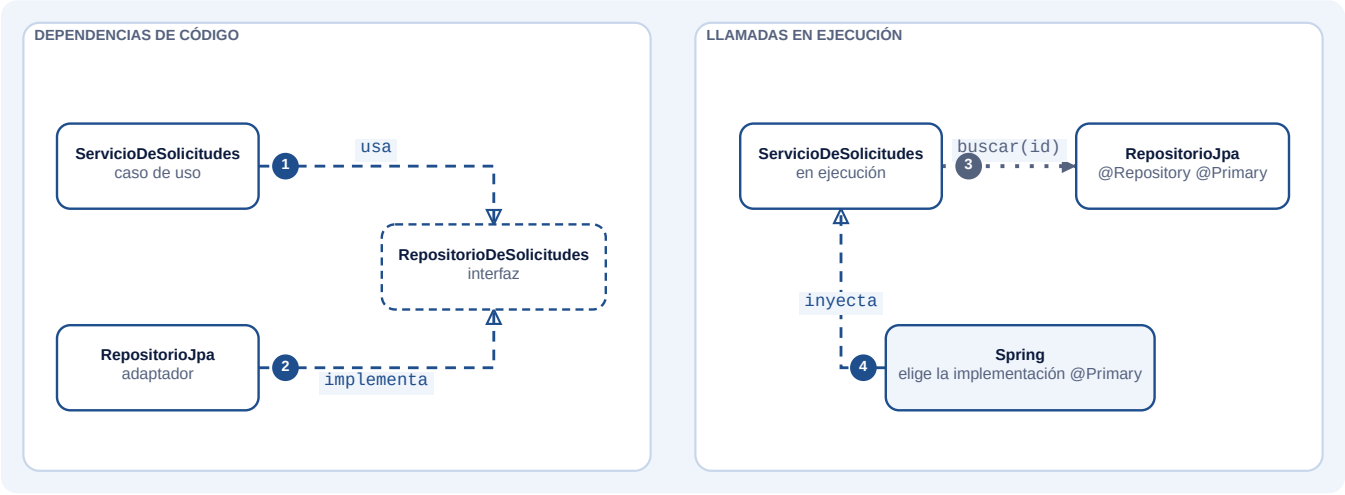
21. Cambiar la pieza sin tocar el negocio

EN PANTALLA

- Hay varios adaptadores para el mismo puerto
- Spring usa el que tiene @Primary
- Hoy: PostgreSQL y publicador con reintentos
- En las pruebas: un repositorio en memoria

Comparación: Como un enchufe: el aparato no sabe de qué planta viene la electricidad, solo que el enchufe cumple la norma.

DIAGRAMA PASO A PASO · DEPENDENCIAS DE CÓDIGO FRENTE A LLAMADAS



1. El servicio de dominio solo conoce el contrato que necesita, no la clase que lo cumple.
2. El adaptador también conoce ese contrato y lo cumple. Los dos dependen del contrato.
3. Cuando la aplicación corre, la llamada sí llega al adaptador.
4. Spring elige cuál usar: el que está marcado con @Primary, hoy el que guarda en PostgreSQL.

Flechas punteadas: dependencias de código. Flecha de puntos finos: llamada interna explicada.

QUÉ DECIR

“Piensen en un enchufe. Un aparato no sabe si la electricidad viene de una hidroeléctrica o de paneles solares. Solo sabe que el enchufe cumple la norma. Los puertos son ese enchufe.”

“Por eso podemos tener varios adaptadores para el mismo puerto. Hoy, Spring usa el que marcamos con @Primary: el que guarda en PostgreSQL y el que publica con reintentos. En las pruebas usamos uno que guarda en memoria, y el negocio ni se entera del cambio.”

“Dejamos las versiones alternativas en el código a propósito, como material de aprendizaje: muestran que cambiar la pieza no obliga a tocar las reglas.”

SI PREGUNTAN

¿La interfaz guarda algo por sí misma?

No. Solo declara el contrato; el adaptador es el que hace el trabajo.

PARA PASAR

“Esa misma idea, llevada más lejos, separa aplicaciones completas.”

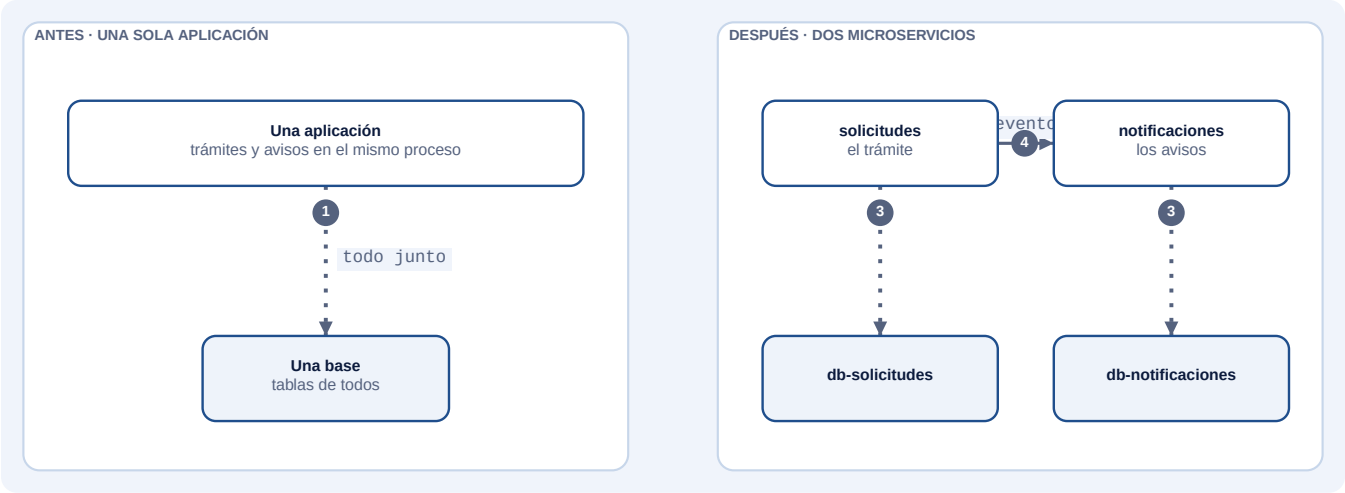
---

22. De monolito a microservicios

EN PANTALLA

- Separamos por responsabilidad
- El trámite y los avisos cambian por motivos distintos
- Cada servicio es dueño de su base
- Se hablan con un contrato: el evento

DIAGRAMA PASO A PASO · DE MONOLITO A MICROSERVICIOS



Paso interno explicado Trabajo posterior

1. Antes: una sola aplicación hace todo y comparte una base.
2. Separamos por responsabilidad: el trámite y los avisos cambian por motivos distintos.
3. Cada servicio es dueño de su base. Ninguno lee la base del otro.
4. Se comunican con un contrato: el evento SolicitudEnviada por HTTP.

Separar tiene costos: red, fallos parciales y datos repartidos. Por eso aparece el outbox.

QUÉ DECIR

“En el taller de descomposición del monolito aprendimos que la pregunta clave no es técnica. Es: ¿qué partes del negocio cambian juntas?”

“Un monolito es una sola aplicación que lo hace todo y comparte una base. En ARKA-Lite, el trámite y los avisos cambian por motivos distintos, así que los separamos en dos servicios, cada uno con su propia base.”

“¿Cómo se hablan, entonces? Con un contrato: un mensaje llamado SolicitudEnviada, que dice qué solicitud se envió. Ese contrato está copiado en los dos proyectos, y si cambia de un lado, hay que cambiarlo del otro.”

PARA PASAR

“¿Siempre conviene separar?”

23. Cuándo conviene separar

EN PANTALLA

- Separar agrega red, bases propias y fallos parciales
- Se decide por negocio, equipos y operación
- En ARKA-Lite separamos para aprender esas fronteras

Criterio	Favorece un monolito	Favorece microservicios
Negocio	Las partes cambian juntas	Responsabilidades claramente distintas
Despliegue	Se publica todo junto	Una parte necesita salir sola
Carga	Pareja	Una parte necesita más recursos
Equipos	Un equipo pequeño	Un equipo dueño de cada parte

QUÉ DECIR

“Separar no es gratis. Ahora hay red entre los servicios, cada uno tiene su base, y puede pasar que uno esté arriba y el otro caído. A eso se le llama fallo parcial.”

“Por eso la decisión se toma con criterios como los de esta tabla: si las partes cambian juntas, si necesitan desplegarse por separado, si un equipo distinto es dueño de cada una.”

“En ARKA-Lite separamos para aprender esas fronteras. Y justamente el fallo parcial es el problema que resolvemos en la siguiente parte.”

SI PREGUNTAN

Si un solo equipo mantiene todo y lo publica junto, ¿qué ganamos separando?

Poco: agregamos red y fallos parciales sin un beneficio claro.

PARA PASAR

“Si un servicio puede fallar mientras el otro funciona, ¿cómo evitamos perder datos?”

4 · Guardar sin perder datos

Presenta: Participante 4 · Base de datos

24. Portada · Guardar sin perder datos

EN PANTALLA

- ¿Qué problema resuelve? Si guardamos la solicitud y justo después falla el aviso, podríamos perder la intención de avisar.
- ¿Cómo funciona? Primero se guarda, después se avisa; y el que recibe ignora lo que ya recibió.
- ¿Cómo se ve? Transacción local -> evento pendiente -> relevador -> receptor idempotente.
- ¿Dónde está en ARKA-Lite? Adaptador JPA, tabla evento, RelevadorDeEventosPendientes,

PublicadorConReintento y ConsumoIdempotente.

- ¿Cómo lo comprobamos? En la UI, pestaña Enviadas: eventos entregados y consumidos, y 0 avisos duplicados al reentregar.

QUÉ DECIR

“Ahora vamos con guardar sin perder datos. Primero, qué problema resuelve.”

25. Un adaptador conectado a una base real

EN PANTALLA

- Cada servicio tiene su base PostgreSQL
- schema.sql crea las tablas al arrancar
- Hibernate revisa que las entidades coincidan
- La entidad JPA no es el objeto del dominio

QUÉ DECIR

“En el taller de modelado y consultas conectamos el adaptador a una base de datos real: PostgreSQL. Para no escribir SQL a mano en cada operación, usamos JPA, que traduce entre nuestros objetos y las tablas.”

“Al arrancar, el servicio ejecuta un archivo llamado schema.sql que crea las tablas si no existen. Después, Hibernate revisa que las clases coincidan con las tablas. Si no coinciden, el servicio no arranca, y eso es bueno: preferimos un error al inicio que datos mal guardados.”

“Un detalle de diseño: la clase que JPA guarda no es la misma que usa el negocio. El adaptador traduce entre las dos, y así el negocio sigue sin saber nada de la base.”

PARA PASAR

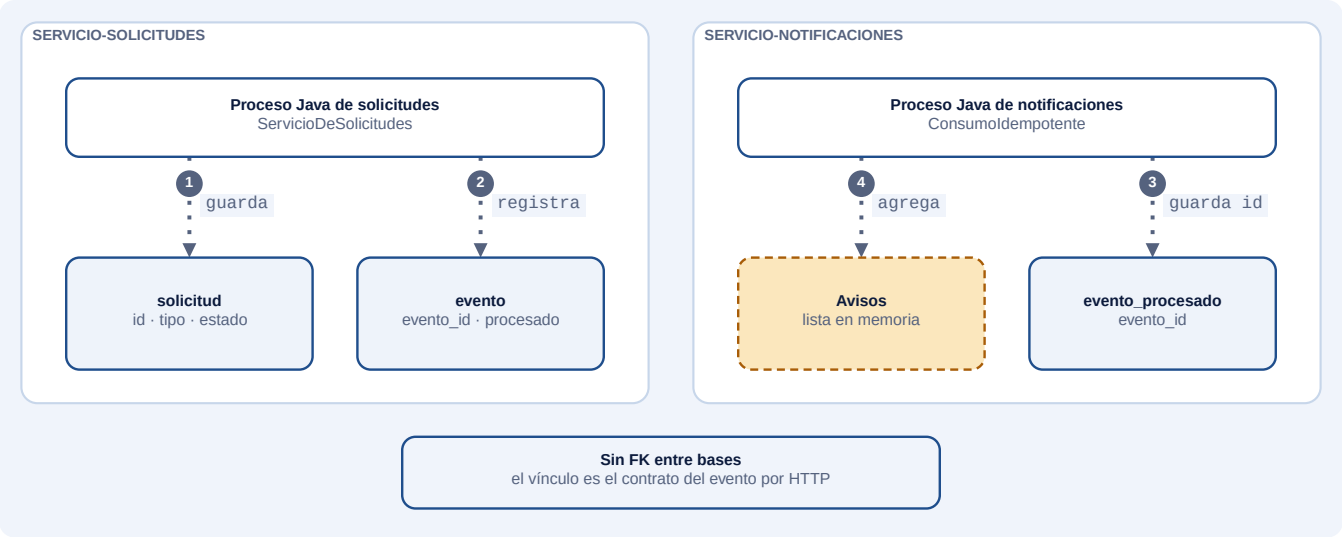
“Ahora veamos el problema más interesante del proyecto.”

26. Dónde vive cada dato

EN PANTALLA

- Solicitudes guarda el trámite y sus eventos
- Notificaciones guarda qué eventos ya recibió
- El texto del aviso vive en memoria
- No hay relación directa entre las dos bases

DIAGRAMA PASO A PASO · DÓNDE VIVE CADA DATO



Paso interno explicado

1. Solicitudes guarda el trámite en su propia base de datos.
2. En esa misma base anota el evento pendiente, para no perder la intención de avisar.
3. Notificaciones guarda en su base los números de evento que ya recibió.
4. El texto del aviso vive en memoria: si el servicio se reinicia, se pierde el texto, pero no el registro.
5. Las dos bases no están conectadas. Lo que las une es el contrato del evento.

Una transacción JPA no revierte una lista Java: si falla el guardado del id, el texto ya quedó en memoria.

QUÉ DECIR

"Antes del problema, ubiquemos los datos. Solicitudes guarda el trámite y una lista de eventos pendientes. Notificaciones guarda la lista de eventos que ya recibió."

"Las dos bases no están conectadas entre sí. Lo único que las une es el contrato del evento."

"Y una aclaración honesta: el texto del aviso vive en memoria. Si notificaciones se reinicia, perdemos ese texto, pero conservamos el registro de qué eventos ya recibimos. Lo mencionamos al final como mejora pendiente."

PARA PASAR

"Con esto claro, veamos el problema de la consistencia."

---

27. El problema de la consistencia

EN PANTALLA · DOS MANERAS, LADO A LADO

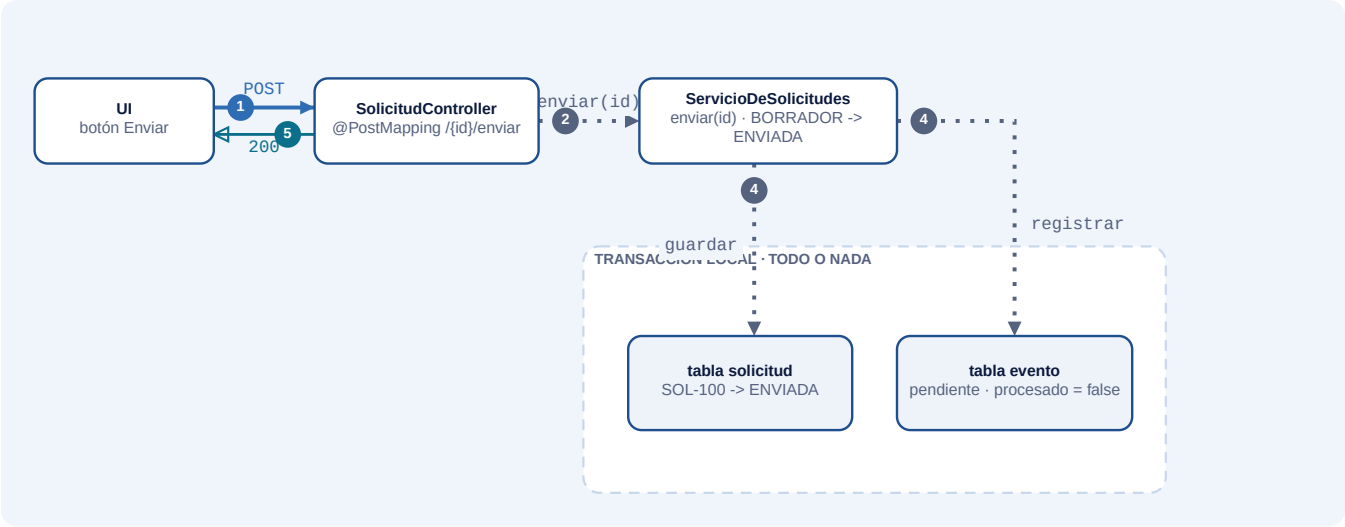
- Avisar en el mismo momento: Se guarda la solicitud como enviada; Justo ahí, notificaciones está caído; El aviso se pierde y nadie se entera
- Guardar primero, avisar después: Solicitud y evento pendiente se guardan juntos; Si notificaciones está caído, el evento espera; El aviso llega cuando el servicio vuelve

IDEAS CLAVE

- Se guardan juntos la solicitud ENVIADA y un evento pendiente
- Si una de las dos falla, no se guarda ninguna

- La respuesta llega enseguida
- El aviso se entrega después

DIAGRAMA PASO A PASO · ENVÍO: ESTADO Y EVENTO EN UNA TRANSACCIÓN



1. La persona pulsa Enviar y la interfaz le pide al servicio que envíe la solicitud.
2. El controlador recibe el pedido y se lo pasa al servicio de dominio.
3. El servicio revisa que la solicitud esté en BORRADOR y la cambia a ENVIADA.
4. En una sola operación se guardan dos cosas: la solicitud ENVIADA y un evento pendiente.
5. La respuesta llega enseguida. El aviso todavía no existe: se entrega después.

Un 200 confirma el envío local. El aviso todavía no existe en ese momento.

QUÉ DECIR

"Imagínense que envían una solicitud justo cuando el servicio de avisos está caído. Si en ese momento intentáramos avisar directamente, la solicitud quedaría enviada, pero nadie se enteraría. Ese es el problema de la consistencia."

"Lo que hicimos fue separar el envío en dos momentos. En el primero, guardamos la solicitud como enviada y, en la misma operación, dejamos anotado que hay un aviso pendiente. Es todo o nada: si una de las dos cosas falla, no se guarda ninguna."

"La respuesta le llega a la persona enseguida. El aviso se entrega en un segundo momento, que vemos en la siguiente lámina."

SI PREGUNTAN

¿Que el envío responda bien significa que el aviso ya existe?

No. Significa que la solicitud y el evento quedaron guardados; el aviso llega después.

PARA PASAR

"¿Quién entrega ese aviso pendiente?"

28. La bandeja de salida

EN PANTALLA · FLUJO PASO A PASO

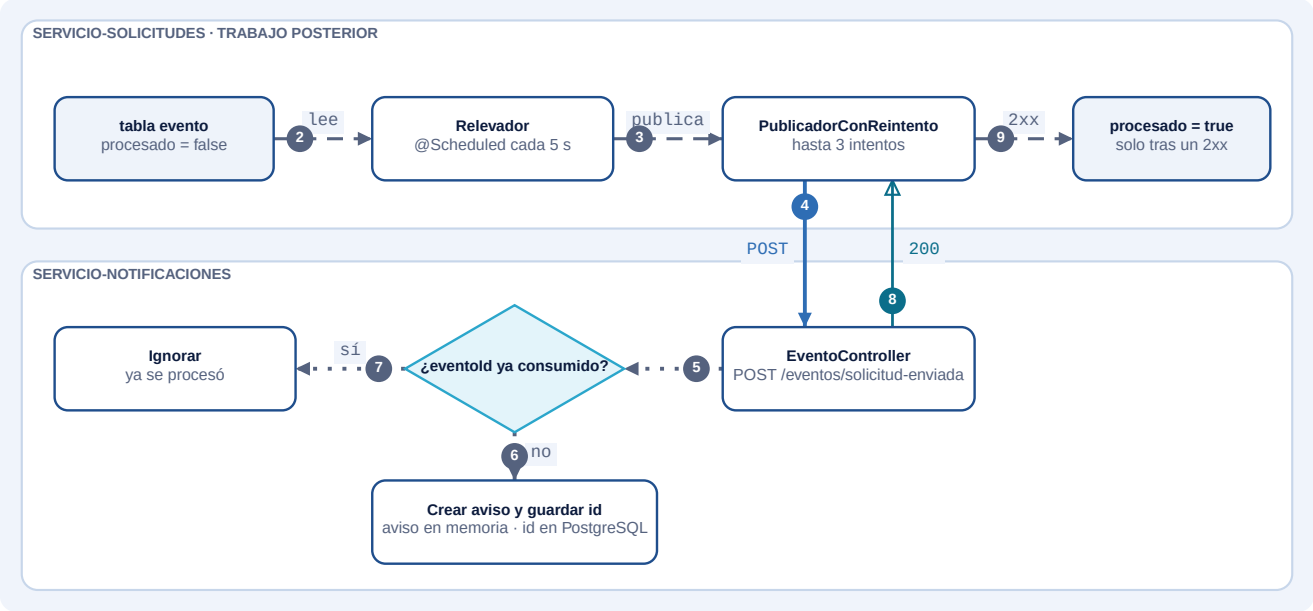
- 1. Evento pendiente: Espera en la tabla evento, junto a la solicitud.
- 2. Relevador: Revisa los pendientes cada 5 segundos.
- 3. Publicador: Lo entrega por HTTP, con hasta 3 intentos.
- 4. Notificaciones: Lo recibe y el evento queda procesado.
- Si si falla en el paso 3: Sigue pendiente: Con el mismo número de evento, se reintenta en el siguiente ciclo.

IDEAS CLAVE

- El evento espera en la tabla de pendientes
- El relevador revisa cada 5 segundos
- Lo entrega con hasta 3 intentos
- Si falla, vuelve a intentar en el siguiente ciclo

Comparación: Como dejar una carta en la bandeja de salida: aunque el mensajero no pase hoy, la carta no se pierde.

DIAGRAMA PASO A PASO · ENTREGA POSTERIOR DEL EVENTO



1. El evento espera en la tabla de pendientes.
2. Cada 5 segundos, el relevador revisa si hay pendientes.
3. El publicador lo entrega hasta con tres intentos, siempre con el mismo número de evento.
4. El evento llega a notificaciones por HTTP.
5. Notificaciones revisa si ya conoce ese número de evento.
6. Si es nuevo, crea el aviso y anota el número.
7. Si ya lo conocía, lo ignora. Así una reentrega no duplica el aviso.
8. Notificaciones responde que lo recibió.
9. Solo entonces el relevador lo marca como entregado.
10. Si esa respuesta se pierde, el evento se vuelve a mandar, y la idempotencia evita el aviso doble.

El outbox no guarda el contexto de traza de la petición original: la entrega aparece como trabajo aparte.

QUÉ DECIR

"Esto se llama patrón outbox, que en español sería bandeja de salida. Es como dejar una carta en la bandeja: aunque el mensajero no pase hoy, la carta no se pierde."

"Nuestro mensajero se llama relevador. Cada cinco segundos revisa la tabla de eventos pendientes y los entrega a notificaciones. Si no logra entregarlos, lo intenta hasta tres veces, y si aun así falla, el evento sigue pendiente para el siguiente ciclo."

"Un detalle importante: cada reintento lleva exactamente el mismo número de evento. Eso es lo que nos permite resolver el siguiente problema."

PARA PASAR

"Si se reintenta, ¿no puede llegar el aviso dos veces?"

---

29. El mismo evento, un solo aviso

EN PANTALLA · FLUJO PASO A PASO

- 1. Llega el evento: Trae su número: el eventoId.
- 2. ¿Ya lo recibimos?: Se busca el número en la tabla evento_procesado.
- 3. Crear el aviso: Solo si el número es nuevo.
- 4. Anotar el número: Para reconocerlo si vuelve a llegar.
- Si ya estaba en el paso 2: Ignorarlo: No se crea otro aviso. Avisos duplicados: cero.

IDEAS CLAVE

- Cada evento tiene un número que no cambia
- Notificaciones anota los números que ya recibió
- Si llega uno repetido, lo ignora
- Lo comprobamos con el botón Reentregar

Comparación: Como un portero con lista de invitados: si alguien ya entró, no lo vuelve a anotar.

QUÉ DECIR

"Sí, puede llegar dos veces. Por ejemplo, si notificaciones recibió el evento pero su respuesta se perdió en el camino, el relevador piensa que falló y lo vuelve a mandar."

"Para eso está la idempotencia, una palabra difícil para una idea sencilla: repetir la misma operación no multiplica su efecto. Notificaciones funciona como un portero con lista de invitados: anota el número de cada evento que recibe, y si llega uno que ya está en la lista, lo ignora."

"En la demostración lo vamos a comprobar: vamos a reentregar a propósito un evento que ya llegó, y vamos a ver que no aparece un aviso nuevo."

QUÉ HACER (NO SE DICE)

- Mostrar: En la UI, pestaña Enviadas: Reentregar un evento ya consumido. Debe verse: El diálogo "El aviso no se duplicó" y Avisos duplicados en 0.

SI PREGUNTAN

¿Qué resuelve el outbox y qué resuelve la idempotencia?

El outbox evita perder el evento; la idempotencia evita duplicar su efecto.

PARA PASAR

“Los datos están cuidados. Ahora, ¿cómo llevamos todo esto a cualquier computadora?”

5 · Empaquetar y entregar

Presenta: Participante 5 · DevOps

30. Portada · Empaquetar y entregar

EN PANTALLA

- ¿Qué problema resuelve? “En mi máquina funciona”: cada computadora tiene otro Java, otras variables y otra red.
- ¿Cómo funciona? Docker empaqueta cada servicio, Compose levanta el sistema y el pipeline prueba cada cambio.
- ¿Cómo se ve? Dockerfile -> imagen -> contenedor; push -> pipeline -> imagen publicada.
- ¿Dónde está en ARKA-Lite? Un Dockerfile por servicio, compose.yaml y .github/workflows/ci.yml.
- ¿Cómo lo comprobamos? docker compose ps con todo sano y la ejecución en GitHub Actions.

QUÉ DECIR

“Ahora vamos con empaquetar y entregar. Primero, qué problema resuelve.”

31. Docker y Dockerfile

EN PANTALLA · FLUJO PASO A PASO

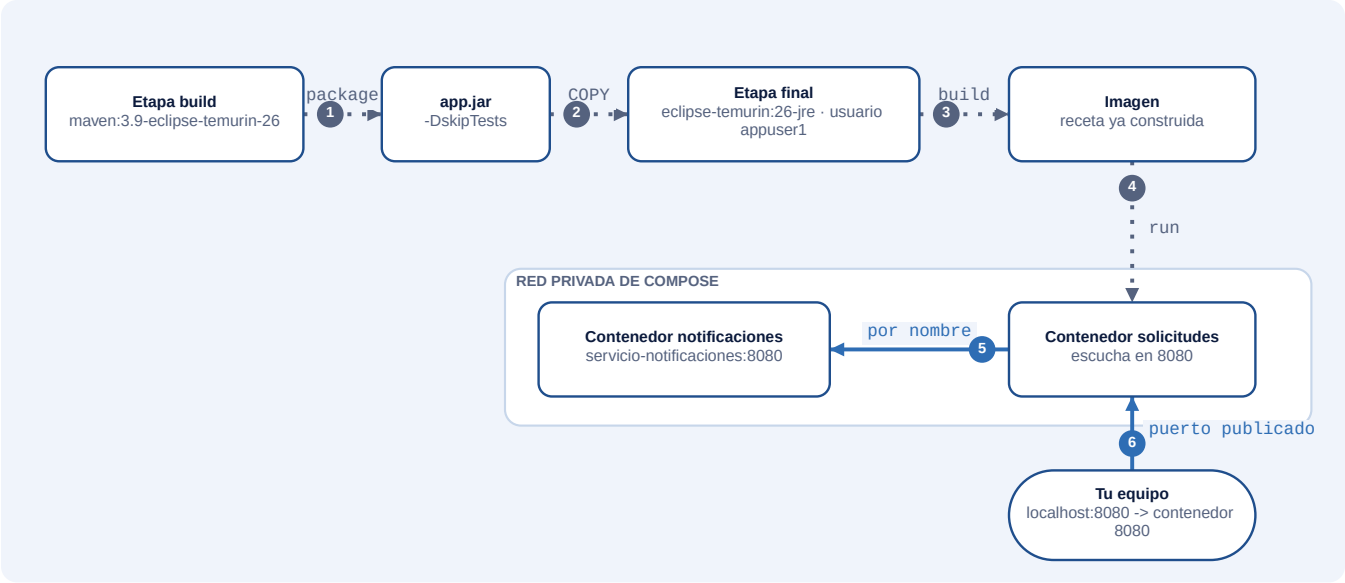
- 1. Dockerfile: La receta: qué instalar y qué copiar.
- 2. Etapa build: Compila el JAR con Maven.
- 3. Imagen: Solo Java y el JAR, con un usuario sin privilegios.
- 4. Contenedor: La imagen corriendo, igual en cualquier computadora.

IDEAS CLAVE

- Dockerfile: la receta
- Imagen: el resultado empaquetado
- Contenedor: la imagen corriendo
- Dos etapas: compilar y ejecutar

Comparación: Como una receta de cocina: con los mismos ingredientes y pasos, sale el mismo plato en cualquier cocina.

DIAGRAMA PASO A PASO · DE DOCKERFILE A CONTENEDOR



Paso interno explicado — Petición

1. La primera etapa tiene Maven y Java completo, y compila el programa.
2. La etapa final solo lleva lo necesario para ejecutarlo, con un usuario sin privilegios.
3. El resultado es una imagen: la misma receta para cualquier computadora.
4. Un contenedor es esa imagen corriendo, con su configuración.
5. Dentro de la red, solicitudes llama a notificaciones por su nombre, no por localhost.
6. Desde tu computadora entras por el puerto publicado.

Construir la imagen no prueba el servicio: el CI ejecuta verify antes de construir.

QUÉ DECIR

"¿Cuántas veces hemos escuchado "en mi máquina funciona"? Docker existe para que eso no vuelva a pasar."

"El Dockerfile es una receta. La imagen es el plato ya preparado y empaquetado. Y el contenedor es ese plato servido: la imagen corriendo. Con la misma receta, sale lo mismo en cualquier computadora."

"En el taller del Dockerfile usamos dos etapas. La primera tiene todas las herramientas para compilar. La segunda solo lleva lo necesario para ejecutar, así la imagen final es más liviana, y corre con un usuario sin privilegios, que es más seguro."

PARA PASAR

"Un contenedor solo no es el sistema. Necesitamos levantarlos todos juntos."

---

32. Docker Compose: el sistema completo

EN PANTALLA

- Servicios, bases, UI y logs en un archivo
- Dentro de la red, cada servicio se llama por su nombre
- Las variables de entorno cambian la configuración
- El puerto publicado no tiene que ser el interno

Quién llama	A quién	Dirección
Navegador	UI	http://localhost:8088
Proxy de la UI	solicitudes	http://servicio-solicitudes:8080
solicitudes	notificaciones	http://servicio-notificaciones:8080
solicitudes	su base	jdbc:postgresql://db-solicitudes:5432/…

QUÉ DECIR

"En el taller de Compose conectamos los dos microservicios. Con un solo comando levantamos todo: los dos servicios, sus dos bases, la interfaz y las herramientas de logs."

"Dentro de la red que crea Compose, cada servicio encuentra al otro por su nombre. Solicitudes no llama a localhost: llama a servicio-notificaciones. Esto es importante, porque dentro de un contenedor, localhost es el mismo contenedor."

"Y la configuración entra por variables de entorno: la dirección de la base, la del otro servicio. Así la misma imagen sirve en nuestra computadora y en el clúster, sin recompilar."

QUÉ HACER (NO SE DICE)

- Mostrar: docker compose ps Debe verse: La UI, los servicios y las bases arriba y sanos.

SI PREGUNTAN

Dentro de un contenedor, ¿a dónde lleva localhost?

Al mismo contenedor, no a otro servicio. Por eso usamos el nombre del servicio.

PARA PASAR

"Ya corre en cualquier computadora. Ahora, cómo validamos cada cambio sin hacerlo a mano."

33. Integración y entrega continua

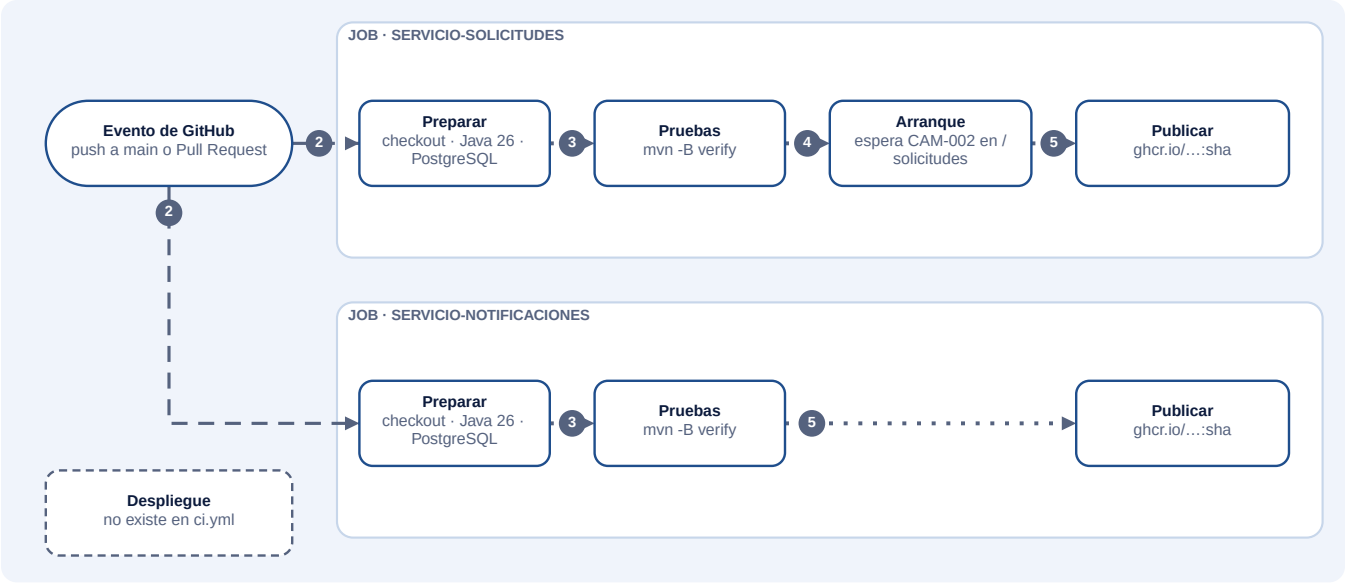
EN PANTALLA · LAS ETAPAS DEL PIPELINE

- 1. Push o PR: Alguien sube un cambio.
- 2. Compilar: Java 26 y Maven, un trabajo por servicio.
- 3. Probar: Todas las pruebas, cada servicio con su base.
- 4. Arrancar: ¿El servicio realmente levanta?
- 5. Imagen: Se construye con Docker.
- 6. Publicar: En GHCR, con el SHA del commit.

IDEAS CLAVE

- Cada push o Pull Request dispara el pipeline
- Compila y prueba cada servicio con su base
- Comprueba que el servicio arranca
- Publica la imagen con el código del commit

DIAGRAMA PASO A PASO · CI/CD CON GITHUB ACTIONS



Trabajo posterior Paso interno explicado

1. Un push a main o un Pull Request dispara el pipeline.
2. Se crea un trabajo por servicio, cada uno con su base de datos de prueba.
3. Cada trabajo compila y corre todas las pruebas del servicio.
4. En solicitudes, además, comprueba que el servicio arranca de verdad.
5. Si todo pasa, construye la imagen y la publica con el código del commit.
6. Publicar no es desplegar: lo que corre en el clúster cambia en un paso aparte.

La publicación también se intenta en PR y puede fallar por permisos, por ejemplo desde un fork.

QUÉ DECIR

"En el taller del pipeline configuramos GitHub Actions para que trabaje por nosotros. Cada vez que alguien sube un cambio o abre un Pull Request, se dispara solo."

"Por cada servicio, prepara Java y una base de datos, compila, corre todas las pruebas y comprueba que el servicio realmente arranca. Si algo falla, nos avisa en minutos."

"Si todo pasa, construye la imagen y la publica con el código del commit. Así cada imagen se puede relacionar con el cambio exacto que la produjo. Integración continua es probar seguido; entrega continua es dejar siempre una versión lista para desplegar."

SI PREGUNTAN

¿Publicar la imagen la despliega en Kubernetes?

No. Desplegar es un paso aparte.

PARA PASAR

"Con cada cambio probado y empaquetado, falta llevarlo al clúster."

6 · Desplegar y operar

Presenta: Participante 6 · DevOps

34. Portada · Desplegar y operar

EN PANTALLA

- ¿Qué problema resuelve? Si un servicio se cae de madrugada, alguien tiene que levantarlo y después averiguar qué pasó.
- ¿Cómo funciona? Kubernetes mantiene el sistema corriendo; los logs y la salud nos dicen qué está pasando.
- ¿Cómo se ve? Deployment -> Pod -> Service; App -> Promtail -> Loki -> Grafana.
- ¿Dónde está en ARKA-Lite? Los manifiestos de k8s/, Promtail, Loki y Grafana, y Actuator en cada servicio.
- ¿Cómo lo comprobamos? kubectl get pods, una búsqueda en Grafana por número de evento y la pestaña Salud de la UI.

QUÉ DECIR

“Ahora vamos con desplegar y operar. Primero, qué problema resuelve.”

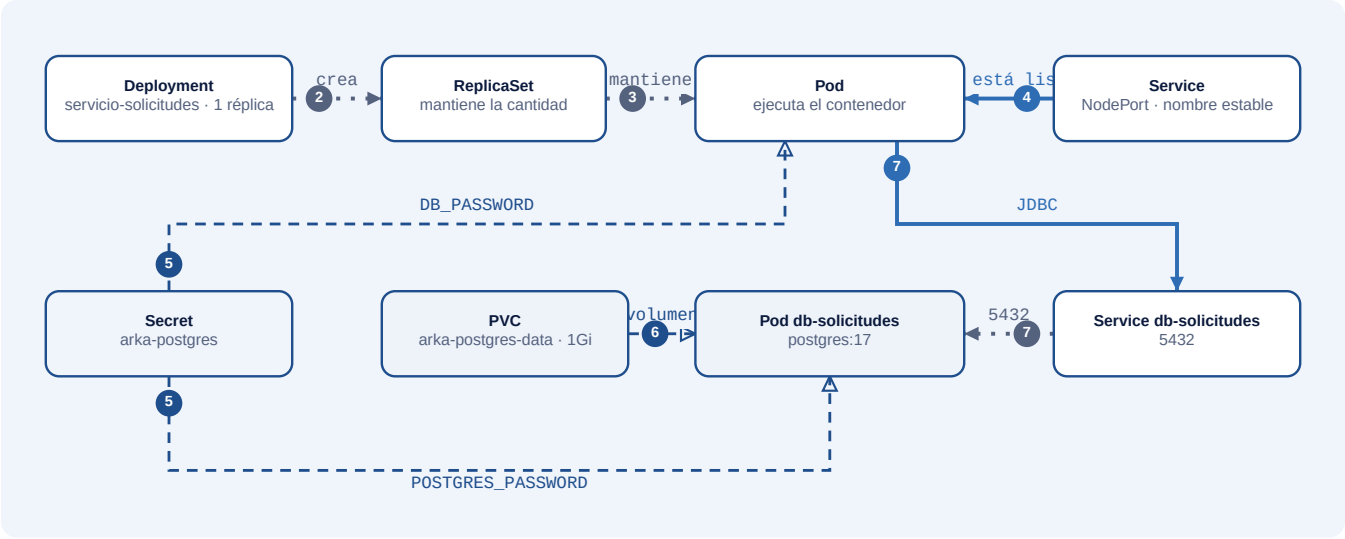
35. Del contenedor al clúster

EN PANTALLA

- Deployment: cuántas copias de cada servicio
- Pod: donde corre el contenedor
- Service: un nombre fijo para encontrarlo
- Secret y disco persistente para las bases

Comparación: Como un termostato: le dices la temperatura que quieres y él se encarga de mantenerla.

DIAGRAMA PASO A PASO · KUBERNETES: ESTADO DESEADO



Paso interno explicado — Petición — Dependencia de código

1. Declaramos un Deployment: qué imagen usar y cuántas copias queremos.
2. Kubernetes mantiene ese número de copias. Si una falta, crea otra.
3. Cada copia corre en un Pod. Antes de arrancar, espera a la base y a notificaciones.
4. El Service da un nombre fijo y solo manda tráfico a los Pods listos.
5. El Secret entrega la contraseña de la base sin escribirla en el código.
6. La base guarda sus datos en un disco que sobrevive a los reinicios. Aun así, no es un respaldo.
7. La aplicación llega a la base por su nombre de Service, no por una dirección fija.

Los manifiestos actuales no declaran Ingress, HPA ni ConfigMap. La UI agrega un ConfigMap propio.

QUÉ DECIR

"Kubernetes funciona como un termostato. No le decimos qué hacer paso a paso: le decimos el estado que queremos, por ejemplo una copia de cada servicio, y él se encarga de mantenerlo."

"El Deployment dice cuántas copias queremos. Cada copia corre en un Pod. El Service le da a cada servicio un nombre fijo dentro del clúster, para que los demás lo encuentren. Y las bases guardan su contraseña en un Secret y sus datos en un disco que sobrevive a los reinicios."

"En el taller desplegamos ARKA-Lite completo en un clúster con un solo comando: kubectl apply sobre la carpeta k8s."

PARA PASAR

"¿Y cómo sabe Kubernetes si un servicio está bien?"

36. Puesta a punto: arranque y salud

EN PANTALLA

- Antes de arrancar, espera a sus dependencias
- Readiness: ¿ya puede recibir tráfico?
- Liveness: ¿hay que reiniciarlo?
- Si un Pod muere, Kubernetes crea otro

Si el Pod está…	Primera pregunta
Pending	¿Hay recursos o falta el disco?
ImagePullBackOff	¿Existe la imagen en el clúster?
CrashLoopBackOff	¿Qué dicen los logs del contenedor?
Running, sin Ready	¿Responde la ruta de salud?

QUÉ DECIR

“En la puesta a punto aprendimos que arrancar no es lo mismo que estar listo. Por eso, antes de iniciar, cada servicio espera a que su base y sus dependencias respondan.”

“Kubernetes hace dos preguntas a cada Pod. La primera: ¿ya puedes recibir tráfico? Si no, no le manda peticiones. La segunda: ¿sigues vivo? Si no responde, lo reinicia.”

“Y si borramos un Pod a propósito, Kubernetes crea otro solo. En la tabla está la primera pregunta que nos hacemos según el estado en que veamos un Pod.”

QUÉ HACER (NO SE DICE)

- Mostrar: kubectl delete pod de servicio-solicitudes y luego kubectl get pods. Debe verse: Aparece un Pod nuevo que vuelve a estar Ready.

PARA PASAR

“El sistema corre. ¿Cómo sabemos qué está pasando adentro?”

---

37. Centralizar los logs

EN PANTALLA · FLUJO PASO A PASO

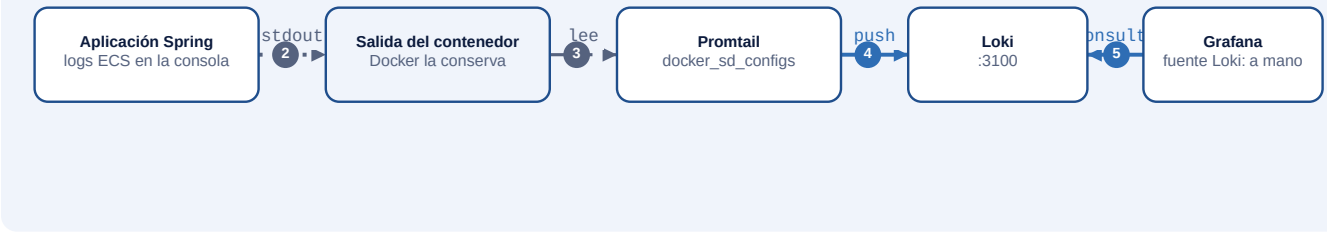
- 1. Servicios: Escriben sus logs en JSON, con el traceId.
- 2. Promtail: Los recoge de todos los contenedores.
- 3. Loki: Los guarda y los ordena por servicio.
- 4. Grafana: Permite buscarlos en un solo lugar.

IDEAS CLAVE

- Cada servicio escribe sus logs en JSON
- Promtail los recoge
- Loki los guarda
- Grafana permite buscarlos

Comparación: Como la caja negra de un avión: registra todo lo que pasó, para revisarlo después.

DIAGRAMA PASO A PASO · RECORRIDO DE LOS LOGS



Paso interno explicado Trabajo posterior Petición

1. Cada servicio escribe sus logs en formato JSON.
2. Docker guarda lo que escribe cada contenedor.
3. Promtail recoge esos logs de todos los contenedores.
4. Loki los guarda y permite buscarlos.

5. Grafana muestra los logs y deja filtrarlos, por ejemplo, por número de evento.

Los logs se escriben en cada servicio. La traza no continúa automáticamente a través del outbox.

QUÉ DECIR

"Los logs son como la caja negra de un avión: registran lo que pasó para poder revisarlo después. El problema es que cada servicio tiene su propia caja negra."

"En el taller de centralización de logs armamos este recorrido. Cada servicio escribe sus logs en formato JSON. Promtail los recoge de todos los contenedores, Loki los guarda y Grafana nos deja buscarlos en un solo lugar."

"Así podemos seguir un evento de un servicio a otro buscando su número, sin entrar a cada contenedor. Desde la interfaz, el menú Contratos tiene un acceso directo a Grafana."

QUÉ HACER (NO SE DICE)

- Mostrar: Grafana ? Explore ? Loki: {service="servicio-solicitudes"} |= "evento". Debe verse: Las líneas "Intentando publicar evento" con el número del evento.

PARA PASAR

"Además de los logs, cada servicio dice si está bien."

---

38. Salud y monitoreo

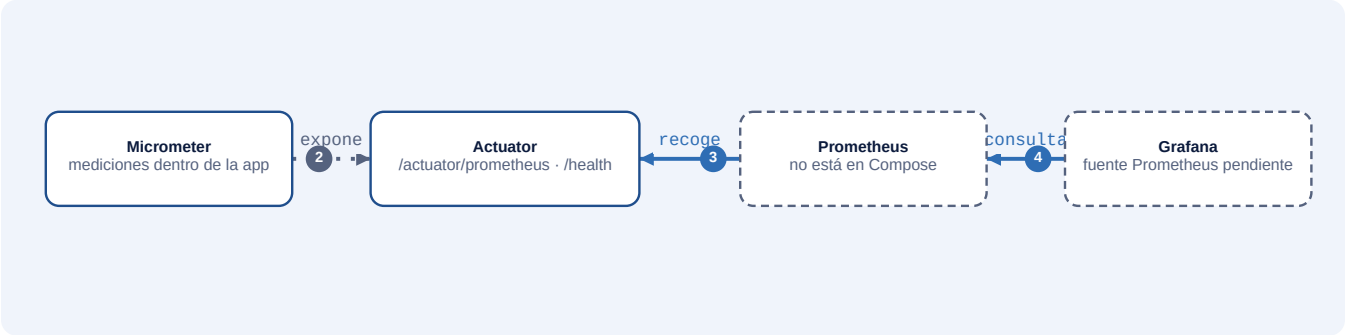
EN PANTALLA · CAPTURAS REALES

- Captura de La pestaña Salud de la UI (localhost:8088/#/salud)

IDEAS CLAVE

- Actuator expone la salud de cada servicio
- Revisa, por ejemplo, la conexión con la base
- Kubernetes y la UI la consultan
- UP no garantiza que todos los avisos llegaron

DIAGRAMA PASO A PASO · RECORRIDO DE LAS MÉTRICAS



Paso interno explicado — Petición

1. Cada servicio mide cosas como cuántas peticiones recibe.
2. Actuator expone esas mediciones y también la salud del servicio.
3. Un servidor Prometheus las guardaría en el tiempo. Hoy queda pendiente.
4. Grafana las mostraría en gráficos. También queda pendiente.

Health resume indicadores configurados; no demuestra que un evento de negocio terminó.

QUÉ DECIR

“Cada servicio tiene una dirección de salud que responde si está bien. Por dentro revisa, por ejemplo, que pueda hablar con su base de datos. Kubernetes la usa para decidir si un Pod está listo, y la interfaz la muestra en la pestaña Salud.”

“Pero ojo: que un servicio diga UP solo significa que responde. No garantiza que todos los avisos llegaron. Para eso miramos la pestaña Enviadas y los logs.”

“Las métricas, como cuántas peticiones recibe cada servicio, ya se exponen. Guardarlas en el tiempo con Prometheus queda como siguiente paso.”

SI PREGUNTAN

Si todo está UP y falta un aviso, ¿qué miramos?

Si el evento se entregó y se consumió, en Enviadas, y los logs del relevador y de notificaciones.

PARA PASAR

“Ya tenemos todas las piezas. Falta ensamblarlas con la interfaz.”

7 · Ensamblaje: la UI

Presenta: Participante 7 · Desarrollo

39. Portada · Ensamblaje: la UI

EN PANTALLA

- ¿Qué problema resuelve? Dos servicios sueltos no son un producto: alguien tiene que poder usarlos y entenderlos.
- ¿Cómo funciona? Una interfaz web consume los dos servicios por un mismo origen y explica cómo funcionan.
- ¿Cómo se ve? Navegador -> /api -> nginx -> solicitudes o notificaciones.
- ¿Dónde está en ARKA-Lite? arka-lite-ui: React, nginx como proxy, DataTables, Tom Select y SweetAlert2.
- ¿Cómo lo comprobamos? La demostración en vivo: crear, enviar, ver el aviso y reentregar sin duplicar.

QUÉ DECIR

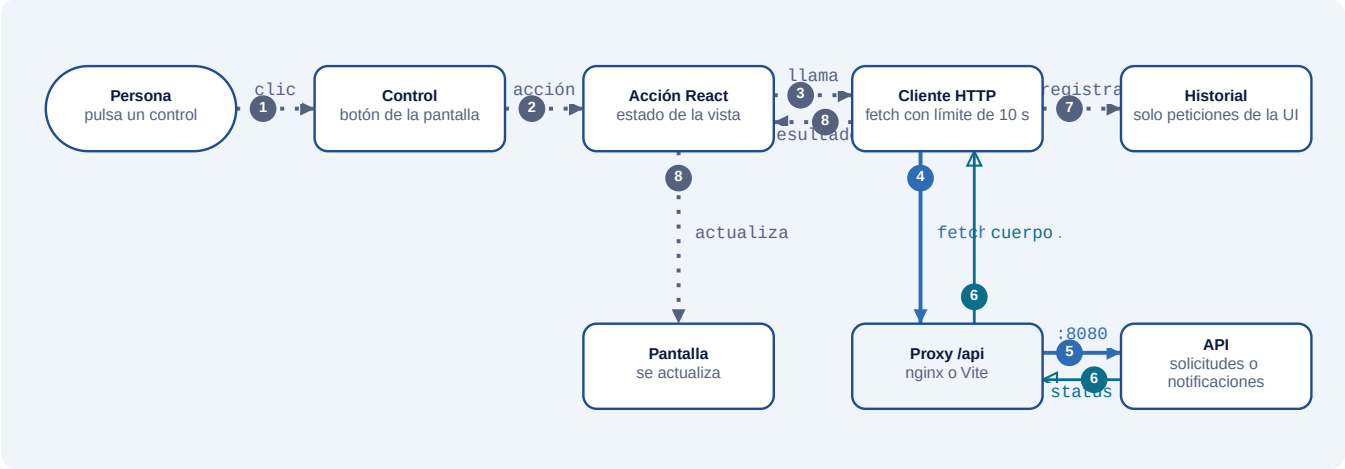
“Ahora vamos con ensamblaje: la ui. Primero, qué problema resuelve.”

40. Cómo la UI consume los dos servicios

EN PANTALLA

- El navegador solo conoce /api
- nginx reenvía cada ruta al servicio que corresponde
- La UI registra cada petición que hace
- Si un POST tarda, no lo repite sola

DIAGRAMA PASO A PASO · CÓMO FUNCIONA LA UI



Paso interno explicado — Petición — Respuesta

1. La persona pulsa un botón en la pantalla.
2. Si la acción cambia datos, la interfaz pide confirmarla antes de enviarla.
3. El cliente HTTP mide el tiempo y, si tarda más de 10 segundos, no lo repite solo.
4. El navegador llama a /api y nginx lo pasa al servicio que corresponde.
5. nginx llega al servicio por su nombre interno.
6. Vuelve la respuesta con su código: 200, 201, 404, 409 o un error si el servicio no responde.
7. La petición queda registrada con su método, ruta, código y duración.
8. La pantalla muestra el resultado con un mensaje claro.

La UI solo ve sus propias peticiones. El tráfico entre servicios no pasa por aquí.

QUÉ DECIR

"La interfaz es una aplicación web hecha con React y servida por nginx. El navegador nunca conoce las direcciones internas de los servicios: siempre llama a /api, en la misma dirección de la página."

"nginx funciona como una recepcionista: recibe cada petición y la pasa al servicio que corresponde. Lo que empieza con /api/solicitudes va a solicitudes; lo de notificaciones, a notificaciones."

"Y un detalle de cuidado: si una operación que cambia datos tarda demasiado, la interfaz no la repite por su cuenta, porque el cambio pudo haberse guardado aunque la respuesta no llegara."

PARA PASAR

"¿Y qué ve la persona cuando abre la interfaz?"

41. Una interfaz simple, con ayuda

EN PANTALLA · CAPTURAS REALES

- Captura de Solicitudes (localhost:8088/#/solicitudes): Crear, ver las tuyas y decidir.
- Captura de Enviadas (localhost:8088/#/enviadas): El recorrido de cada evento y Reentregar.
- Captura de Salud (localhost:8088/#/salud): Los dos servicios, Swagger y Grafana.
- Captura de Ayuda (localhost:8088/#/ayuda): Diagramas paso a paso de cómo funciona.

IDEAS CLAVE

- Solicitudes: crear, ver las tuyas, enviar, aprobar o rechazar
- Enviadas: el recorrido de cada evento y Reentregar
- Salud: validar los dos servicios
- Ayuda: diagramas paso a paso de cómo funciona

QUÉ DECIR

"La interfaz tiene solo cuatro pestañas. En Solicitudes creas una solicitud, ves las que creaste y tienes a la mano las acciones que corresponden a cada una. Antes de cada acción, la app te pide confirmar."

"En Enviadas ves el recorrido de cada evento: registrado, entregado, consumido y aviso. En Salud validas que los dos servicios estén bien, y desde ahí también abres los contratos en Swagger y los logs en Grafana."

"Y en Ayuda está la explicación: cada tema responde una pregunta, como qué pasa cuando envío una solicitud, con un diagrama que avanza paso a paso. Los colores de la plantilla se pueden personalizar desde la propia app."

PARA PASAR

"Ahora sí, vamos a verlo funcionando."

---

42. Demostración en vivo

EN PANTALLA · FLUJO PASO A PASO

- 1. Ver la salud: Los dos servicios en verde.
- 2. Crear: Una solicitud nueva: DEMO-01.
- 3. Enviar: Y ver llegar el evento y el aviso.
- 4. Reentregar: El aviso no se duplica.

IDEAS CLAVE

- Ver los dos servicios en verde
- Crear una solicitud nueva
- Enviarla y ver llegar el evento
- Reentregarlo: el aviso no se duplica

QUÉ DECIR

"Abrimos la interfaz. Arriba a la derecha vemos que los dos servicios están en verde. Creamos una solicitud nueva, con un identificador propio. La app confirma que se creó y la marca como nuestra."

"Ahora la enviamos. La app nos pide confirmar, confirmamos, y en la pestaña Enviadas vemos el evento registrado. En unos segundos, el relevador lo entrega, notificaciones lo recibe y aparece el aviso. Fíjense en la línea fina bajo la barra superior: se ilumina cuando un evento termina su viaje."

"Y la prueba final: reentregamos a propósito el mismo evento. La app nos confirma que notificaciones lo reconoció y no creó otro aviso. Avisos duplicados: cero. Eso es la idempotencia, funcionando."

QUÉ HACER (NO SE DICE)

- Usar un identificador que no exista, por ejemplo DEMO-01. Si ya existe, la app muestra el mensaje 409: explicarlo y usar otro.
- Si el evento tarda, esperar: la pestaña Enviadas se actualiza sola cada 5 segundos.
- Mostrar: Solicitudes: crear DEMO-01 y enviarla. Enviadas: esperar el aviso y pulsar Reentregar. Debe verse: 201 al crear; evento entregado y consumido; 1 aviso; “El aviso no se duplicó”.

PARA PASAR

“Para cerrar, repasemos lo que aprendimos.”

Cierre

Presenta: Participante 1 · Desarrollo

43. Lo que aprendimos, en ARKA-Lite

EN PANTALLA

- Cada tema del curso tiene un lugar en el proyecto
- Del primer endpoint al clúster
- Con pruebas y una demostración que lo respaldan

Tema del curso	Dónde está en ARKA-Lite
Spring Boot y endpoints REST	Los controladores de solicitudes y notificaciones
Manejo global de errores	ManejadorGlobalDeErrores: 404, 409 y 400
Git, Swagger y pruebas	Ramas y Pull Requests, Swagger, JUnit y MockMvc
Puertos y adaptadores	Los paquetes dominio y adaptadores, con @Primary
De monolito a microservicios	Dos servicios, cada uno con su base
Docker y Compose	Dockerfile en dos etapas y compose.yaml
Integración y entrega continua	El pipeline de GitHub Actions
Kubernetes	Deployments, Services, Secret y disco persistente
Base de datos y consistencia	JPA, bandeja de salida, relevador e idempotencia
Logs y monitoreo	Promtail, Loki, Grafana y la salud de cada servicio
Ensamblaje del sistema	La interfaz: Solicitudes, Enviadas, Salud y Ayuda

QUÉ DECIR

“Esta tabla resume el curso entero. Cada tema que vimos tiene un lugar concreto en ARKA-Lite, desde el primer endpoint con Spring Boot hasta el despliegue en Kubernetes y la centralización de logs.”

“Y no lo decimos solo en una lámina: lo acaban de ver funcionando, con pruebas automáticas que lo vigilan y un pipeline que lo valida en cada cambio.”

PARA PASAR

“También queremos ser honestos con lo que le falta.”

---

44. Lo que le falta

EN PANTALLA

- El texto de los avisos vive en memoria
- No hay inicio de sesión: es un laboratorio
- Sin mensajería dedicada ni métricas históricas
- El pipeline publica, pero no despliega solo

QUÉ DECIR

“ARKA-Lite es un laboratorio, y lo decimos con claridad. El texto de los avisos se pierde si notificaciones se reinicia. No tiene inicio de sesión y las contraseñas son de prueba. No usa un sistema de mensajería dedicado, no guarda métricas en el tiempo y el pipeline publica las imágenes, pero no las despliega solo.”

“Los siguientes pasos serían guardar el aviso en la base junto con su número de evento, agregar autenticación y sacar las contraseñas del repositorio.”

PARA PASAR

“Gracias por su atención.”

---

45. Gracias

EN PANTALLA

- Equipo arka-litos · Upskilling Backend Java
- ¿Preguntas?

QUÉ DECIR

“Empezamos con un endpoint y terminamos con un sistema de dos microservicios, empaquetado, desplegado, observado y con una interfaz que cualquiera puede usar para comprobar cómo funciona.”

“Muchas gracias por acompañarnos, de parte de todo el equipo arka-litos. Con gusto respondemos sus preguntas.”